

10th October, 2026

The Challenge

Challenge name/tagline: Fresh eyes on fresh produce!

Challenge one-liner: use the ceiling cameras of an autonomous store to detect, count and check the fruits and vegetables on each gondola against the planogram.

Introduction

Fruits and vegetables are the most demanding section of a store for computer vision. Products are sold loose, with no barcode or packaging. They are piled rather than aligned, they change shape, size and color as they ripen, and they are constantly hidden by the hands of shoppers and staff.

At the same time, it is the section where knowing the state of the shelf matters most, since produce is perishable:

- Empty gondola » lost sales » unhappy customers
- Overstocked gondola » products left too long on the shelf » waste
- Wrong product in the wrong place » confused customers » wrong prices

Today, these checks depend on staff walking the store. Labelling this kind of data is itself hard and subjective: counting bananas in a pile is a challenge even for a person. Across hundreds of stores and thousands of cameras, nobody can label every image, every day.

So, the solution must learn to see the shelf without being told what is right. Only a small, hidden validation set has been annotated, so that all solutions can be compared fairly.

The core of the challenge

One of the first steps towards a store that knows its own shelves is to recognise, count and check the products on each gondola, continuously and without human help.

Our company operates autonomous stores fitted with ceiling cameras and has created a dataset specifically for this challenge. It contains images from 14 cameras installed over the Fruits & Vegetables section of Continente Bom Dia S. Romão in Leiria — the largest autonomous store in the world, powered by Sensei — extracted during one full day of operation. Each camera sees one or multiple gondola modules.

Gondolas and products were chosen so that products can be recognised generically (e.g. banana, orange), with no need to know brands or packaging. The file planogram.json holds the planogram active at the start of the day for each module (for each product, its id, its name and the image coordinates of its planogram area).

The planogram file

planogram.json tells, for each of the 14 cameras, which product should occupy each area of the shelf at the start of the day. It describes 152 planogram areas in total, with this structure:

```
{
  "Frutas e Legumes": {
    "cameras": {
      "6215": {
        "products": [
          {
            "area_id": "6215_2",
            "product_id": "11908",
            "product_name": "BANANA CAT I CNT KG",
            "image_coordinates": {
              "top_left": [299, 350],
              "top_right": [464, 347],
              "bottom_right": [479, 438],
              "bottom_left": [331, 438]
            }
          }
        ]
      }
    }
  }
}
```

- **Frutas e Legumes:** the store section.
- **cameras:** one entry per camera, keyed by the camera id (e.g. 6215).
- **products:** the list of planogram areas seen by that camera, each with its area_id (a unique id made of the camera id and an area number, e.g. 6215_2), product_id, product_name and image_coordinates.
- **image_coordinates:** the four corners of the area in the camera image, as [x, y] pixel positions measured from the top-left corner of the image. Areas are four-sided shapes rather than rectangles, because of the camera perspective.

A few details are worth keeping in mind:

- The same product can occupy several areas. For example, BANANA CAT I CNT KG has six areas on camera 6215, so stock should be counted per area and then added up per product.
- Some coordinates are negative, meaning the area extends beyond the edge of the image and is only partly visible (e.g. the top shelf of camera 6383).
- Product names are in Portuguese, as in the store's product catalogue. Names ending in KG are sold loose, by weight, so a unit is a single fruit or vegetable. Names with a weight (e.g. 500 G, EMB. 3KG) are packaged, so a unit is one package.

The goal is to produce an application that, for each image, solves as many of the following tasks as feasible:

1. **Detection and classification:** find the products in the image and name them.
2. **Stock counting:** estimate how much of each product is in each planogram area, as an item count or a fill level.
3. **Planogram comparison:** say, for each area, whether it is compliant, low, empty or holding the wrong product.

The images are split into three sets. The train set and the test set are shared with candidates and have no annotations. The validation set is kept hidden from candidates during the training and programming stage, and is the only set with ground-truth annotations. When requested, candidates run their own code on the validation images and submit the outputs, which are then compared automatically with the annotations.

The annotation format

The ground truth of the validation set is stored in `annotations.json`. It covers 26 images, one or two per camera, and every planogram area in each image (289 annotated areas in total), with this structure:

```
{
  "6215": {
    "1751052241.jpg": {
      "6215_2": {
        "planogram_compliant": true,
        "stock_level": "half",
        "stock_value": 0.5
      }
    }
  }
}
```

- **Camera id** (e.g. 6215): the same ids as in `planogram.json`.
- **Image file name** (e.g. 1751052241.jpg): the capture time of the image, as a Unix timestamp.
- **area_id** (e.g. 6215_2): the planogram area, exactly as in `planogram.json`, where its product and image coordinates can be found.
- **planogram_compliant**: true if the product actually shown in the area is the one the planogram expects, false otherwise.
- **stock_level**: how much product is in the area, in words: empty, half or full. The same level is given as a number in `stock_value`: 0 for empty, 0.5 for half and 1 for full.

Candidates submit their output in exactly the same format: one entry per image and per area_id, with planogram_compliant, stock_level and stock_value filled in by their application. To compare an output with the ground truth, each area is matched by camera id, image file name and area_id, and then planogram_compliant and stock_value are compared as described below.

How the solution will be appraised

The outputs on the validation set are compared automatically with the annotations, area by area. The scoring follows the way a member of the store staff would use the application.

Picture an employee starting a shift. They open the app, see the stock level of every gondola and every planogram mismatch, and plan their work from it: what to restock first, what to move back to its place, what to leave alone. Every wrong piece of information has a cost:

- False alarm » staff walk to a shelf that needed nothing » time lost
- Missed problem » empty or wrong shelf left in front of customers » lost sales
- Repeated errors » frustration » staff stop trusting the app

The scoring therefore rewards information staff can act on and penalises each error by the trouble it would cause them.

The level of achievement will be measured area by area, adding and subtracting points according to the consequences for the staff¹:

1. Product identification of each planogram area, compared with planogram_compliant: the answer is correct when it states rightly whether the product actually shown in the image is the one the planogram expects:
 - a. Correct identification; +1 point.
 - b. Wrong identification; -2 points, since staff go there to arrange products and there is nothing to arrange.
2. Stock level of each planogram area, compared with stock_value (0 for empty, 0.5 for half, 1 for full): a lower value than the annotation is below the real shelf stock, a higher value is above it:
 - a. Correct stock level; +1 point.
 - b. Stock level below the real shelf stock; -2 points, since staff go there to replenish and the shelf is already full.

- c. Stock level above the real shelf stock; -3 points, since the shelf runs empty without anyone noticing and sales are lost.

In addition, the user interface and user experience (UI/UX) of the application will be evaluated qualitatively, on a scale from 1 to 5, judging how clearly and quickly a member of the store staff can see what needs to be done. This evaluation is used only in case of a tie in points between candidates: among them, the one with the best UI/UX ranks first.

¹ The score is the total of points over all observed areas: each area can earn up to 2 points, 1 for identification and 1 for stock level, so the maximum on the validation set is $289 \times 2 = 578$ points. An area that is annotated but missing from the output counts as a wrong identification (-2) and a stock level above the real stock (-3), since staff get no warning about it. Assume a solution is run on all 289 planogram areas of the validation set:

Case	Areas	Points each	Points
Identification correct	270	+1	+270
Identification wrong	19	-2	-38
Stock level correct	230	+1	+230
Stock level below the real stock	35	-2	-70
Stock level above the real stock	24	-3	-72
Total			+320

The score would be 320 points, out of a maximum of 578.

Future potential and benefits

Sensei operates a fleet of autonomous stores, each with cameras over every gondola. A solution that needs no labels can be switched on in a new store with no extra hardware and no labelling effort.

Creating a tool that could "watch" every gondola, all day long, could provide an insight into:

- when each product needs restocking, before the gondola runs empty
- which products stay too long on the shelf and end up as waste
- how well each store follows its planogram, without manual audits

- how each product sells, hour by hour and store by store
- how the gondola layout affects what customers pick

Since most modern stores already have ceiling cameras, this proposal could extend well beyond autonomous stores. To some extent, the same methods might also be used for other loose products in the future, such as bakery or flowers.